

Chapter 2 Assembly Language Programming

Instruction Formats (Opcodes, Mnemonics and Operands)

An instruction is a binary pattern designed inside a microprocessor to perform a specific function. Each instruction has two parts: one is task to be performed, called the operation code (opcode) and second is the data to be operated on, called the operand. The operand or data can be specified in various ways. It may include 8-bit or 16-bit data, an internal register, a memory location, or 8-bit or 16-bit address. In some instructions, the operand is implicit.

Every instruction in 8085 is represented by an 8-bit binary value. A microprocessor executes instruction given by the user. Instruction should be in a language known to the microprocessor. But Microprocessor understands the language of 0's and 1's. This language is called Machine Language. We write 8085 program in assembly language which microprocessor cannot understand so, a program known as Assembler is used to convert an assembly language.

Instruction formats are divided into three types:

- ❖ One-byte instructions: A 1-byte instruction includes the opcode and the operand in the same byte. For example MOV C, A and ADD B.
- ❖ Two-byte instructions: In a 2-byte instruction, the first byte specifies the operation code and a second byte specifies the operand. For example MVI A,08H.
- ❖ Three-byte instructions: In a 3-byte instruction, the first byte specifies the opcode and the following two bytes specify the 16-bit address. Note that the second byte is the low-order address and the third byte is the higher-order address. For example JMP 2000H, LDA D000H.

8085 Instruction Sets

The complete set of instructions which a particular computer can interpret and execute is known as Instruction Set. The program is written using the instructions from this instruction set and the computer fetches and executes these instructions.

These instructions can be classified into the following five functional categories:

- ❖ Data transfer (copy) Instructions
- ❖ Arithmetic Instructions
- ❖ Logical Instructions
- ❖ Branching Instructions
- ❖ Machine-control Instructions

Data Transfer Instructions: This instruction copy the contents of the source register into the destination register. For example:

- I. MOV Rd, Rs → e.g., MOV C, A where C is the destination register and A is source Register. In the below example, Rs and Rd are all general purpose registers such as A, B, C, D, E, H, L. The contents of the source register do not change.

Before Execution

A=08	Flag Register
B	C=00
D	E
H	L
SP	
PC	

After Execution

A=08	Flag Register
B	C=08
D	E
H	L
SP	
PC	

- II. MVI R, data → e.g., MVI A, 05H where 05 is the immediate data and A is the destination register. Here, the immediate data 05H is copied to A register.

A=00	Flag Register
B	C
D	E
H	L
SP	
PC	

A=05	Flag Register
B	C
D	E
H	L
SP	
PC	

- III. MOV M, Rs → e.g., MOV M, C where C is the source register and M is the memory pointer which is HL pair. The HL pair contains the memory address whose data is 00H (assume). C register content is copied to HL pair content.

Before Execution

A	Flag Register
B	C=08
D	E
H(C0)	L(00)
SP	
PC	

FFFFH	
C000H	00
0000H	

After Execution

A	Flag Register
B	C=08
D	E
H(C0)	L(00)
SP	
PC	

FFFFH	
C000H	08
0000H	

- IV. MOV Rd, M → MOV C, M where C is the destination register and M is the memory pointer which is HL pair. This case is quite reverse of the last case.

Before Execution

A	Flag Register
B	C=00
D	E
H=C0	L=00
SP	
PC	

After Execution

A	Flag Register
B	C=08
D	E
H=C0	L=00
SP	
PC	

[C000H]

- V. $MVI\ M, data \rightarrow MVI\ M, 10H$ where M is the memory pointer which is HL pair. This instruction moves immediate data to memory. Now, by taking the address from the HL pair, the immediate data is written to this memory location.

Before execution

A	Flag Register
B	C
D	E
H(C0)	L(00)
SP	
PC	

FFFF	
C000	00
0000	

After execution

A	Flag Register
B	C
D	E
H(C0)	L(00)
SP	
PC	

FFFF	
C000	10
0000	

Here the memory contents are changed.

- VI. $LXI\ Rp, data \rightarrow LXI\ H\ 3000H$ where 3000H is the 16-bit data and H stands for HL pair. This instruction loads register pair with the 16-bit data. Finally we can see that H contains 30H and L contains 00H.

Before execution

A	Flag Register
B	C
D	E
H(05)	L(00)
SP	
PC	

After execution

A	Flag Register
B	C
D	E
H(30)	L(00)
SP	
PC	

- VII. LDA address → LDA D000H where D000H is the address. This instruction loads the accumulator direct from memory location D000H.

Before execution

A(50)	Flag Register
B	C
D	E
H	L
SP	
PC	

FFFF	
D000	20
0000	

After execution

A(20)	Flag Register
B	C
D	E
H	L
SP	
PC	

FFFF	
D000	20
0000	

- VIII. STA address → STA D001H where D001H is the address. This instruction stores the accumulator direct to memory location D001H.

Before execution

A(50)	Flag Register
B	C
D	E
H	L
SP	
PC	

FFFF	
D001	20
0000	

After execution

A(50)	Flag Register
B	C
D	E
H	L
SP	
PC	

FFFF	
D001	50
0000	

IX. LHL D address → LHL D001H where D001H is the address. This instruction loads the HL pair direct from memory.

Before execution

A(50)	Flag Register
B	C
D	E
H(D0)	L(01)
SP	
PC	

FFFF	
D001	20
D002	30
0000	

After execution

A(50)	Flag Register
B	C
D	E
H(30) ←	L(20) ←
SP	
PC	

FFFF	
D001	20
D002	30
0000	

X. SHLD address → SHLD D001H where D001H is the address. This instruction stores the HL pair content direct to memory location D001H& D002H.

Before execution

A(50)	Flag Register
B	C
D	E
H(05)	L(04)
SP	
PC	

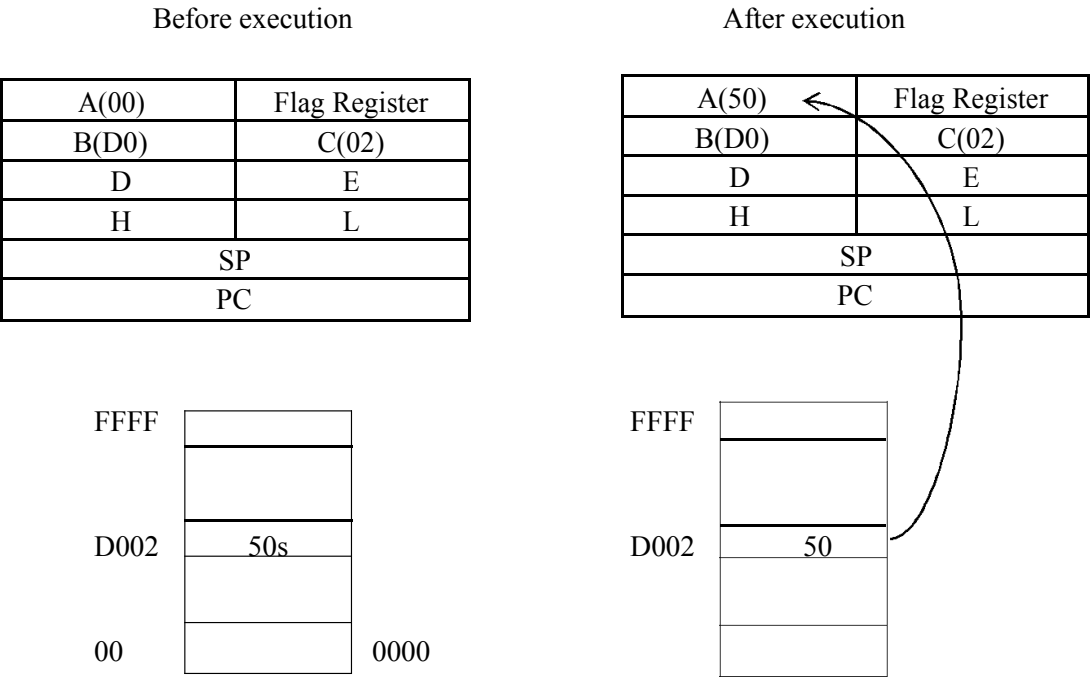
FFFF	
D001	20
D002	30
0000	

After execution

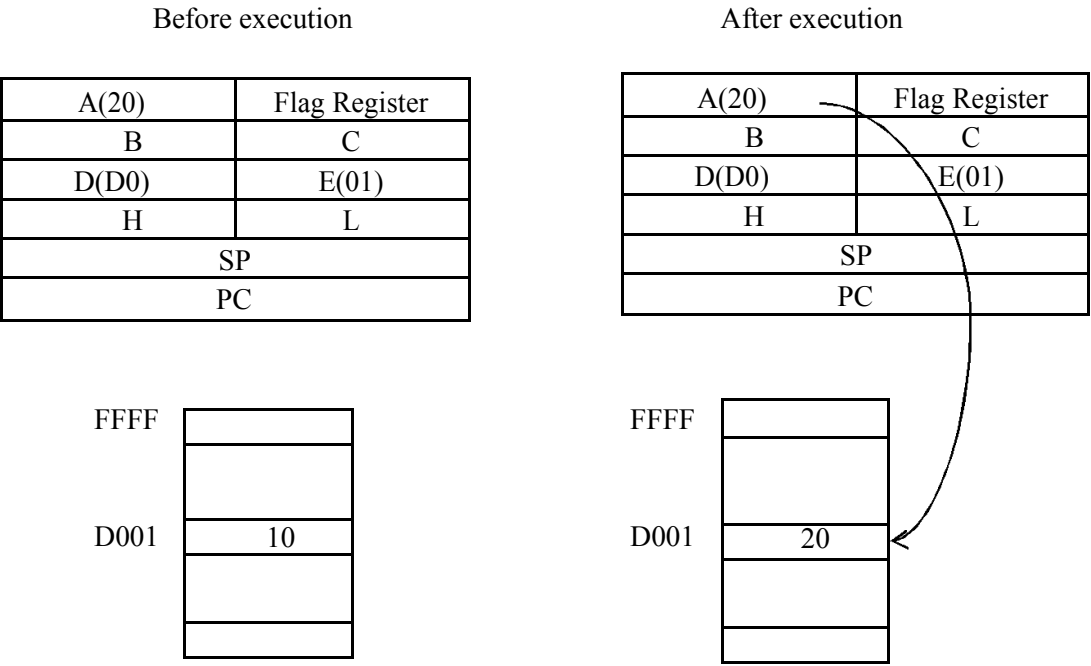
A(50)	Flag Register
B	C
D	E
H(05)	L(04)
SP	
PC	

FFFF	
D001	04
D002	05
0000	

XI. LDAX Rp → LDAX B where B is the Register pair BC. This instruction load accumulator indirect by using a memory pointer.



XII. STAX Rp → STAX D where D is the DE registers pair. This instruction stores the accumulator indirect to memory pointer.



- XIII. XCHG: This instruction is used to exchange the contents of HL with DE pair. This is one byte instruction.

Before execution		After execution	
A		A	
B	C	B	C
D(40)	E(50)	D(20)	E(30)
H(20)	L(30)	H(40)	L(50)
SP		SP	
PC		PC	

Addressing Modes of 8085

The method by which the address of source of data or the address of destination of result is given in the instruction is called Addressing Modes. The term addressing mode refers to the way in which the operand of the instruction is specified. Intel 8085 uses the following addressing modes:

- ❖ Direct Addressing Mode
- ❖ Register Addressing Mode
- ❖ Register Indirect Addressing Mode
- ❖ Immediate Addressing Mode
- ❖ Implicit Addressing Mode

Direct Addressing Mode: In this mode, the address of the operand is given in the instruction itself. For example:

LDA 2500H; H Load the contents of memory location 2500 H in accumulator.

- ❖ LDA is the operation.
- ❖ 2500 H is the address of source.
- ❖ Accumulator is the destination.

Register Addressing Mode: In this mode, the operand is in general purpose register. For example:

MOV A, B; Move the contents of register B to A.

- ❖ MOV is the operation.
- ❖ B is the source of data.
- ❖ A is the destination.

Register Indirect Addressing Mode: In this mode, the address of operand is specified by a register pair. For example:

MOV A, M; Move data from memory location specified by H-L pair to accumulator.

- ❖ MOV is the operation.
- ❖ M is the memory location specified by H-L register pair.
- ❖ A is the destination.

Immediate Addressing Mode: In this mode, the operand is specified within the instruction itself. For example:

MVI A, 05H; Move 05 H in accumulator.

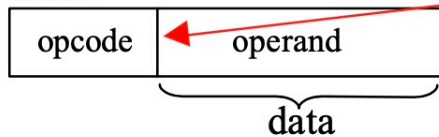
- ❖ MVI is the operation.
- ❖ 05 H is the immediate data (source).
- ❖ A is the destination.

Implicit Addressing Mode: If address of source of data as well as address of destination of result is fixed, then there is no need to give any operand along with the instruction. For example:

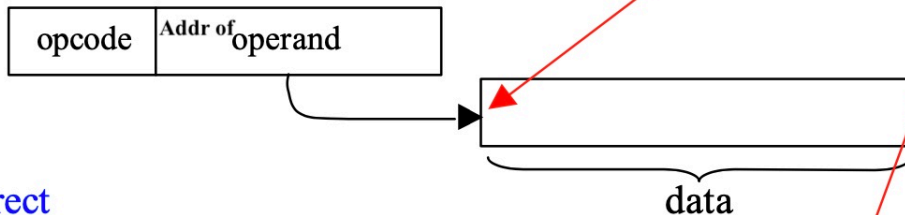
CMA; Complement accumulator.

- ❖ CMA is the operation.
- ❖ A is the source.
- ❖ A is the destination.

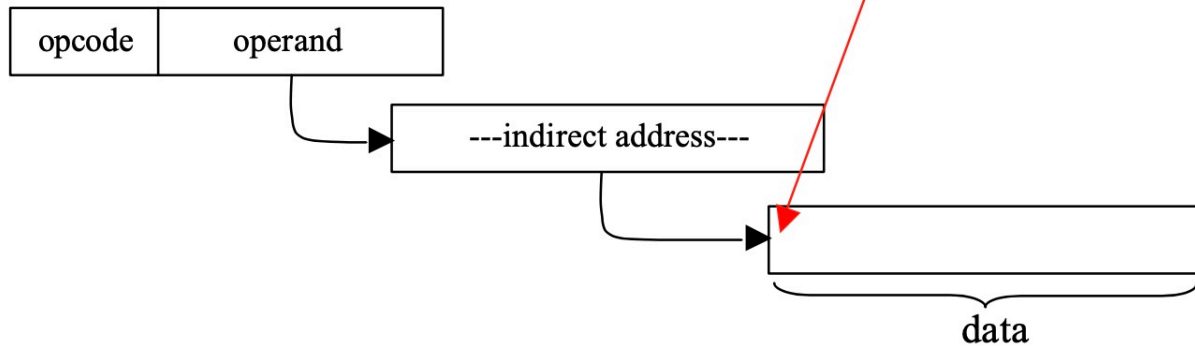
immediate



direct



indirect



Arithmetic Instructions: This group of instructions performs arithmetic operations such as addition, subtraction, increment and decrement.

- I. ADD Rs → e.g., ADD B where A is the destination register and B is source Register. In the above example, Rs is all general purpose registers such as A, B, C, D, E, H, L. The contents of the source register do not change. $A(98) \leftarrow A(47) + B(51)$, The register A value is added with register B value and the result goes to A register.

Before Execution

A=47	S	Z	AC	P	CY
B=51	C				
D	E				
H	L				
SP					
PC					

After Execution

A=98	1	0	0	0	0
B=51	C				
D	E				
H	L				
SP					
PC					

- II. ADD M → e.g., ADD M where M is the memory pointer which is HL pair. The HL pair contains the memory address whose data is 20H (assume). A register value 10 is added with [HL] and the results can be seen in the accumulator.

Before Execution

A=10	S	Z	AC	P	CY
B	C				
D	E				
H(C0)	L(00)				
SP					
PC					

After Execution

A=30	0	0	0	0	0	1	0	0
B	C							
D	E							
H(C0)	L(00)							
SP								
PC								

FFFFH	
C000H	20
0000H	

FFFFH	
C000H	20
0000H	

- III. ADC R $\rightarrow$ ADC B where B is the source register. Here the register B value is added with Accumulator and carry flag CY.

Before Execution

A=3F	S	Z	AC	P	CY
B=20	C				
D	E				
H	L				
SP					
PC					

After Execution

A=60	0	0	0	1	0	1	0	0
B=20	C							
D	E							
H	L							
SP								
PC								

- IV. ADC M $\rightarrow$ ADC M where M is the memory pointer which is HL pair. This instruction adds memory content with Accumulator and Carry flag CY. The final result goes to the accumulator.

Before execution

A=10	S	Z	AC	P	CY
B	C				
D	E				
H(C0)	L(02)				
SP					
PC					

After execution

A=31	0	0	0	0	0	0	0	0
B	C							
D	E							
H(C0)	L(02)							
SP								
PC								

FFFF	
C002	20
0000	

FFFF	
C002	20
0000	

- Before execution
After execution

A=10	0 0 0 1 0 0 0 1
B	C
D	E
H	L
SP	
PC	

- Before execution
After execution

A=E0	1 0 0 0 0 0 0 0
B	C
D	E
H	L
SP	
PC	

- Before execution
After execution

A	0	0	0	0	0	0	0	0
B=20	C=35							
D	E							
H=A0	L=7A							
SP								
PC								

- VIII. SUB R $\rightarrow$ SUB B where B is the source register. This instruction subtracts the contents of register B from the contents of accumulator and the result is placed in accumulator. The contents of B are not altered. All fags are modified to reflect the result. The subtraction is performed by using 2's complement method. If carry flag CY is set the result is negative and is in 2's complement form. If CY is reset, the result is positive and is in normal form.

Before execution

A=37H	S	Z	AC	P	CY
B=40H	C				
D	E				
H	L				
SP					
PC					

After execution

A=F7H	1	0	0	0	0	0	0	1
B=40H	C							
D	E							
H	L							
SP								
PC								

In the example above, no carry is generated so CY=0. The microprocessor complements the carry which means CY=1 which represents that the result is negative and is in 2's complement form.

- IX. SUB M $\rightarrow$ SUB M where M is the memory pointer. This instruction subtracts the memory location contents from accumulator.

Before execution

A=50	S	Z	AC	P	CY
B	C				
D	E				
H(C2)	L(00)				
SP					
PC					

After execution

A=30	0	0	0	0	0	1	0	0
B	C							
D	E							
H(C2)	L(00)							
SP								
PC								

FFFF	
C200	20
C001	
0000	

FFFF	
C200	20
C201	
0000	

- X. SBB R $\rightarrow$ SBB B where B is the Source Register. This instruction subtracts register B and borrow flag from accumulator.

Before execution

A=37	S	Z	AC	P	CY
B=3F	C				
D	E				
H	L				
SP					
PC					

After execution

A=F7	1	0	0	0	0	0	0	1
B=3F	C							
D	E							
H	L							
SP								
PC								

- XI. SBB M $\rightarrow$ SBB M where M is the memory pointer. This instruction subtracts memory contents and borrow flag from accumulator.

Before execution

A=20H	S	Z	AC	P	CY
B	C				
D	E				
H=C2	L=00				
SP					
PC					

After execution

A=D0	1	0	0	0	0	0	0	1
B	C							
D	E							
H=C2	L=00							
SP								
PC								

FFFF	
C200	4F
0000	

FFFF	
C200	4F
0000	

- XII. SUI Data → SUI 50 where 50H is the immediate data. This instruction subtracts immediate data from accumulator.

Before execution

A=20H	S	Z	AC	P	CY
B	C				
D	E				
H	L				
SP					
PC					

After execution

A=D0	1	0	0	0	0	0	0	1
B	C							
D	E							
H	L							
SP								
PC								

- XIII. SBI Data → SBI 4F where 4F is immediate data. This instruction subtracts immediate data and borrow flag from accumulator.

Before execution

A=20H	S	Z	AC	P	CY
B	C				
D	E				
H	L				
SP					
PC					

After execution

A=D0	1	0	0	0	0	0	0	1
B	C							
D	E							
H	L							
SP								
PC								

- XIV. Decimal Adjust Accumulator (DAA):

- If the value of the low order 4-bits D3-0 in the accumulator is greater than 9 or if AC flag is set, the instruction adds 6 to the low order 4-bits of accumulator.
- If the value of the higher order 4-bits D7-4 in the accumulator is greater than 9 or if carry flag is set, the instruction adds 6 to the higher order 4-bits of accumulator.

If we want to add two BCD numbers 12 and 39 and want result in BCD form.

MVI A, 12
ADI 39
DAA

When DAA is executed, it checks lower D0-D3 is greater than 9 or AC flag is set. Here the result addition is 4B in hexadecimal where B is greater than 9 so 6 is added to the lower order 4-bits. However, nothing is added to the higher D4-7 bits. So, the final result 51.

- XV. INR R/DCR R $\rightarrow$ INR B/DCR B where B is source register. This instruction increments/decrements the register value by 1. The below example is for increment.

Before execution

A	S	Z	AC	P	CY
B=2F	C				
D	E				
H	L				
SP					
PC					

After execution

A	0	0	0	1	0	1	0	0
B=30	C							
D	E							
H	L							
SP								
PC								

- XVI. INR M/DCR M where M is the memory pointer. This instruction increment/decrement memory contents by one. The below example is for increment.

Before execution

A	S	Z	AC	P	CY
B	C				
D	E				
H=C2	L=00				
SP					
PC					

After execution

A	0	0	0	0	0	1	0	0
B	C							
D	E							
H=C2	L=00							
SP								
PC								

C200 05

C200 06

XVII. INX Rp/DCX Rp $\rightarrow$ INX B/DCX B where BC is the register pair Rp. This instruction increments/decrements the register pair value by 1. The below example is for increment.

Before execution

A	S	Z	AC	P	CY
B=02	C=FF				
D	E				
H	L				
SP					
PC					

After execution

A	0	0	0	1	0	1	0	0
B=03	C=00							
D	E							
H	L							
SP								
PC								

Logical Instructions: This group of instructions performs logical operations such as and, or, xor etc. with the contents of accumulator.

- I. ANA R $\rightarrow$ e.g., ANA B where A is the destination register and B is source Register. In the below example, Rs is all general purpose registers such as A, B, C, D, E, H, L. The contents of the source register do not change. $A(02) \leftarrow A(56) \text{ AND } B(82)$, The register A value is anded with register B value and the result goes to A register.

Before Execution

A=56	S	Z	AC	P	CY
B=82	C				
D	E				
H	L				
SP					
PC					

After Execution

A=02	0	0	0	1	0	0	0	0
B=82	C							
D	E							
H	L							
SP								
PC								

- II. ANA M $\rightarrow$ e.g., ANA M where M is the memory pointer which is HL pair. The HL pair contains the memory address whose data is F2H (assume). A register value 10 is added with [HL] and the results can be seen in the accumulator.

Before Execution

A=4A	S	Z	AC	P	CY
B	C				
D	E				
H(C0)	L(00)				
SP					
PC					

After Execution

A=42	0	0	0	1	0	1	0	0
B	C							
D	E							
H(C0)	L(00)							
SP								
PC								

FFFFH	
C000H	F2
0000H	

FFFFH	
C000H	F2
0000H	

- III. ANI Data $\rightarrow$ ANI 0FH where A is the destination register. This instruction logically AND the immediate data with accumulator.

Before Execution

A=F7	S	Z	AC	P	CY
B	C				
D	E				
H	L				
SP					
PC					

After Execution

A=07	0	0	0	1	0	0	0	0
B	C							
D	E							
H	L							
SP								
PC								

- IV. ORA $R \rightarrow$ e.g., ORA B where A is the destination register and B is source Register. In the below example, Rs is all general purpose registers such as A, B, C, D, E, H, L. The contents of the source register do not change. A (B7) $\leftarrow$ A (A2) or B (B5), The register A value is anded with register B value and the result goes to A register.

Before Execution

A=A2	S	Z	AC	P	CY
B=B5	C				
D	E				
H	L				
SP					
PC					

After Execution

A=B7	1	0	0	1	0	1	0	0
B=B5	C							
D	E							
H	L							
SP								
PC								

- V. $\text{ORA } M \rightarrow \text{ORA } M$ where M is the memory pointer which is HL pair. This instruction does logical OR with memory content.

Before execution

A=AA	S	Z	AC	P	CY
B	C				
D	E				
H(C0)	L(02)				
SP					
PC					

After execution

A=FF	1	0	0	0	0	1	0	0
B	C							
D	E							
H(C0)	L(02)							
SP								
PC								

VIII. XRA M $\rightarrow$ XRA M where M is the memory pointer which is HL pair. This instruction does Exclusive-OR with memory content.

Before execution

A=A5	S	Z	AC	P	CY
B	C				
D	E				
H(C2)	L(00)				
SP					
PC					

FFFF	
C200	50
C001	
0000	

After execution

A=F5	1	0	0	0	0	1	0	0
B	C							
D	E							
H(C2)	L(00)							
SP								
PC								

FFFF	
C200	50
C201	
0000	

IX. XRI data $\rightarrow$ XRI 2FH where 2FH is the 8-bit data. This instruction does Exclusive-OR of immediate data with accumulator.

Before execution

A=75	S	Z	AC	P	CY
B	C				
D	E				
H	L				
SP					
PC					

After execution

A=5A	0	0	0	0	0	1	0	0
B	C							
D	E							
H	L							
SP								
PC								

- X. $CMA \rightarrow CMA$ where A is the Source Register. This instruction complements the accumulator value and the result is placed in the accumulator.

Before execution

A=AA	S	Z	AC	P	CY
B	C				
D	E				
H	L				
SP					
PC					

After execution

A=55	0	0	0	0	0	0	0	0
B	C							
D	E							
H	L							
SP								
PC								

- XI. CMC: This instruction complements the carry flag. If carry flag is set, the instruction will reset it and carry flag is reset, the instruction will set it. Only the carry flag is complemented. No other flags are modified.
- XII. STC: This instruction sets the carry flag. It does not consider previous carry flag status. Only sets the carry flag. Only the carry flag is set and no other flags are changed.
- XIII. CMP R $\rightarrow$ CMP B where Accumulator value A is compared with B register value. The operation of comparing is performed by subtracting register from accumulator. The contents of register or accumulator are not changed.
The result of comparison by setting flags as follows:
If accumulator > register B: CY is set and Zero Flag is Reset.
If accumulator < register B: Zero Flag is set
If accumulator = register B: CY is set.

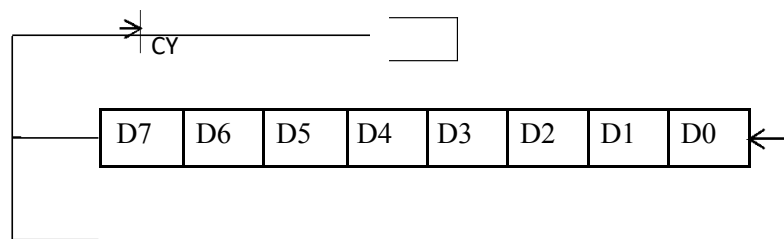
Before execution

A=20H	S	Z	AC	P	CY
B=10H	C				
D	E				
H	L				
SP					
PC					

After execution

A=20H	0	0	0	0	0	0	0	0	0
B=10H	C								
D	E								
H	L								
SP									
PC									

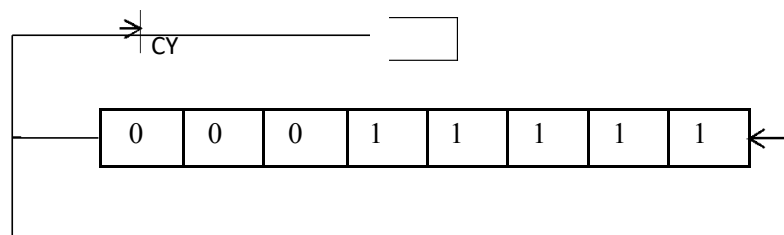
- XIV. **CMP M:** This instruction compares the memory content with the accumulator. The address of the memory location is given by HL register pair. The operation is performed in same way as CMP R instruction. Contents of accumulator and contents of memory location are not changed.
- XV. **CPI Data:** This instruction compares the immediate data with the accumulator similar to that of CMP R and the result is indicated by Z and CY flags. The accumulator contents are not altered.
- XVI. **RLC:** RLC refers to Rotate Accumulator Left. This instruction rotates the content of accumulator left by 1-bit which implies it shifts the bits left by single position. D0 will be transferred to D1 and D2 to D3 and so on. The D7 result is transferred to CY flag and D0. This can be seen in the figure below.



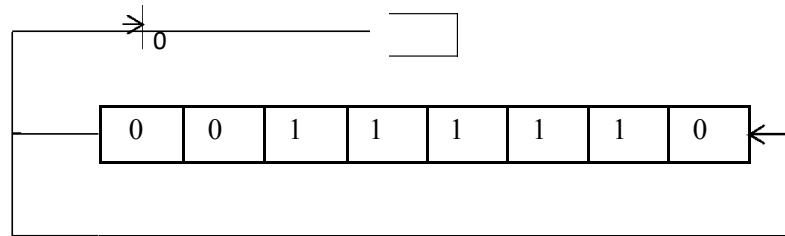
For Example: MVI A, 1F

RLC

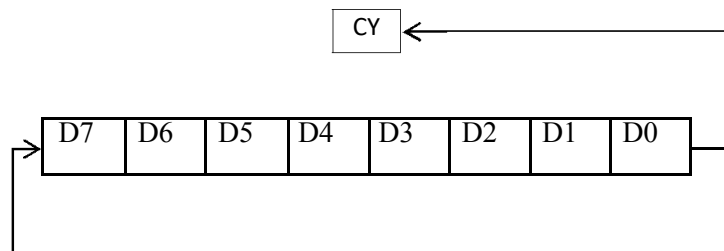
Before Execution



After Execution



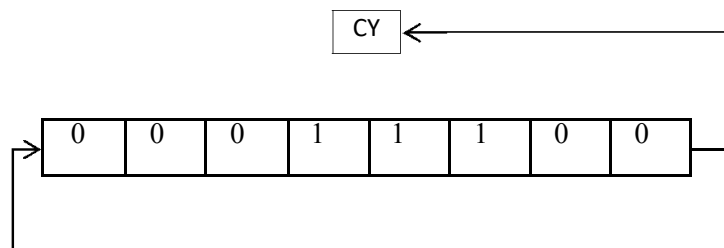
- XVII. RRC: RRC refers to Rotate Accumulator Right. This instruction rotates the content of accumulator right by 1-bit which implies it shifts the bits right by single position. D7 will be transferred to D6 and D6 to D5 and so on. The D0 result is transferred to CY flag and D7. This can be seen in the figure below.



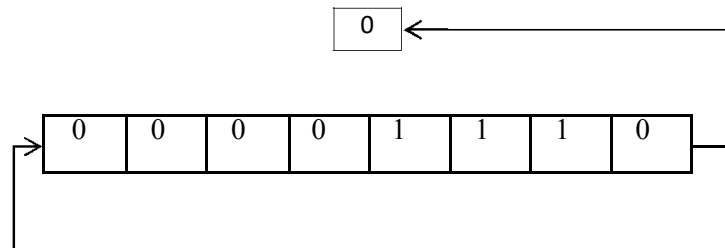
For Example:

MVI A, 1C
RRC

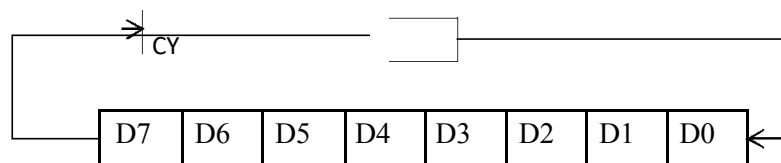
Before Execution



After Execution



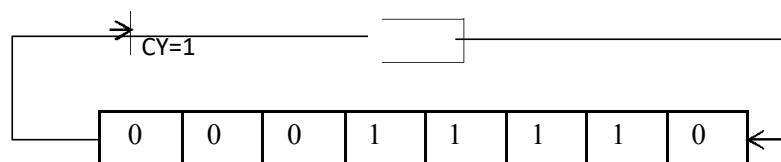
- XVIII. RAL: RAL refers to Rotate Accumulator Left through Carry. This instruction rotates the content of accumulator left by 1-bit. D0 will be transferred to D1 and D1 to D2 and so on. The D7 result is transferred to CY flag and CY flag result is transferred to D0. This can be seen in the figure below.



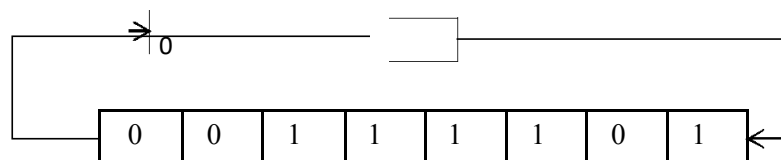
For Example:

MVI A, 1E
RAL

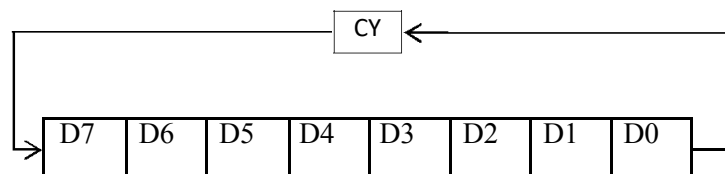
Before Execution



After Execution



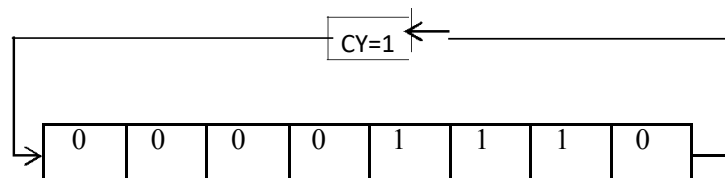
- XIX. RAR: RAR refers to Rotate Accumulator Right through Carry. This instruction rotates the content of accumulator right by 1-bit. D7 will be transferred to D6 and D6 to D5 and so on. The D0 result is transferred to CY flag and CY flag result is transferred to D7. This can be seen in the figure below.



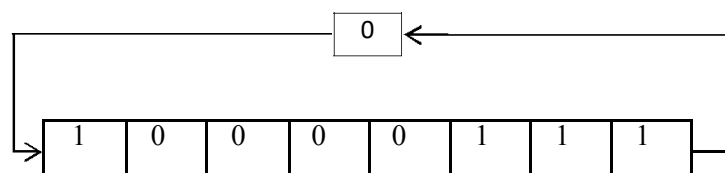
For Example:

MVI A, 0E
RAR

Before Execution



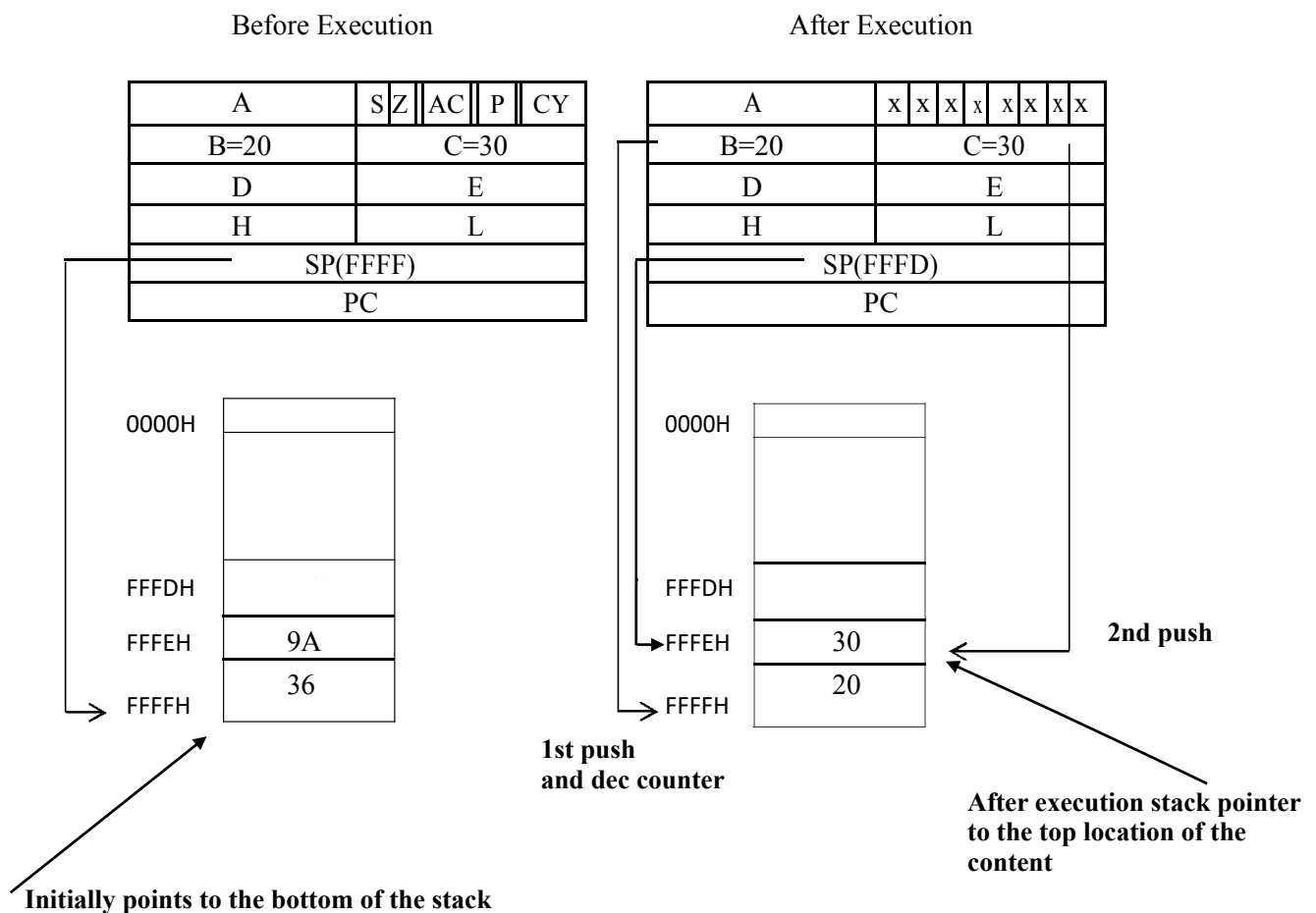
After Execution



Program Control Instructions: This group of instructions includes instruction related to Stack Control and Branch Group.

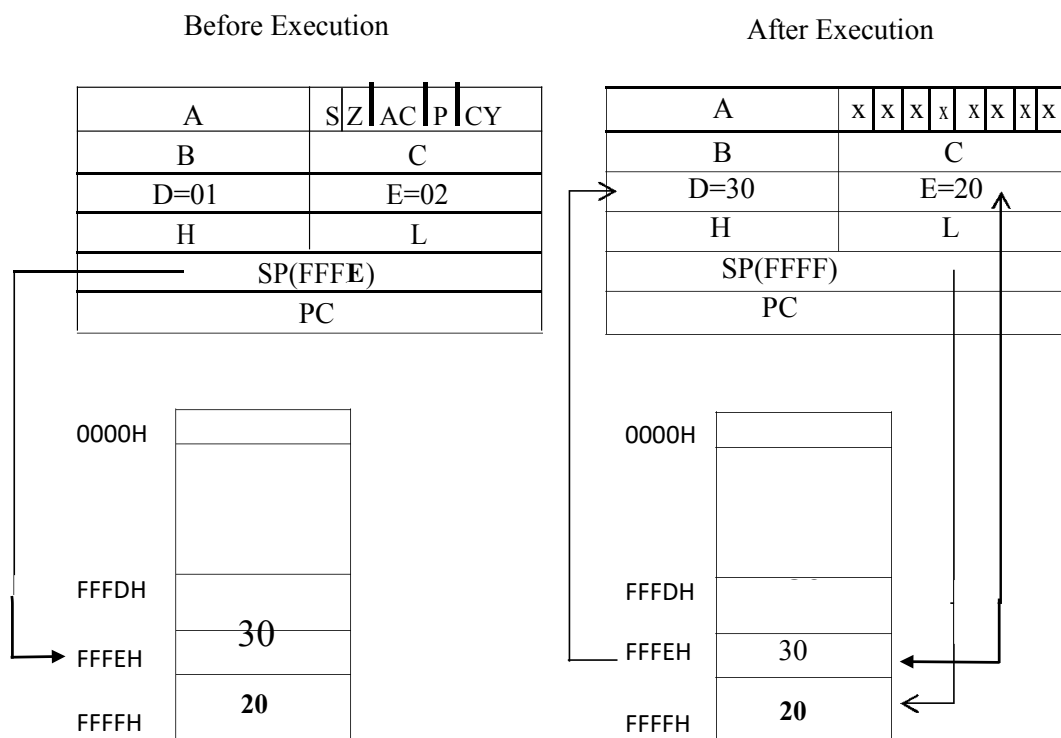
Stack: It is group of memory locations in RAM defined by user. It stores data temporarily in case there is shortage of available general purpose registers. To store the data in this stack we push the contents of register on to stack and these register would be free to be used. After that function is completed, the previous contents are popped from stack and stored in the register. Similar to Program Counter, Stack Pointer (SP) is the memory pointer in the stack. It uses the concept of LIFO. While Pushing the data in the stack area, the SP decrements after each write. Similarly, while Popping increments the address first then pops the data.

- I. PUSH Rp $\rightarrow$ e.g., PUSH B where B is the register pair BC. This instruction pushes the contents of register pair on to the stack. The SP pushes the content of B register to the address pointed by Stack Pointer.

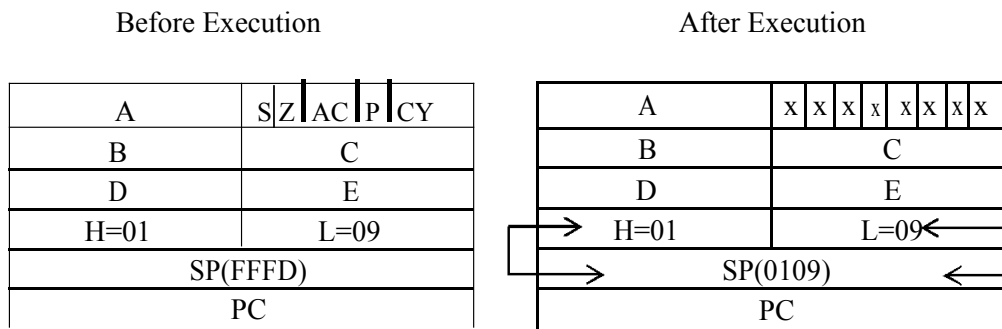


****Note: Stack pointer always points to the data that is at the top of the stack after push operation.**

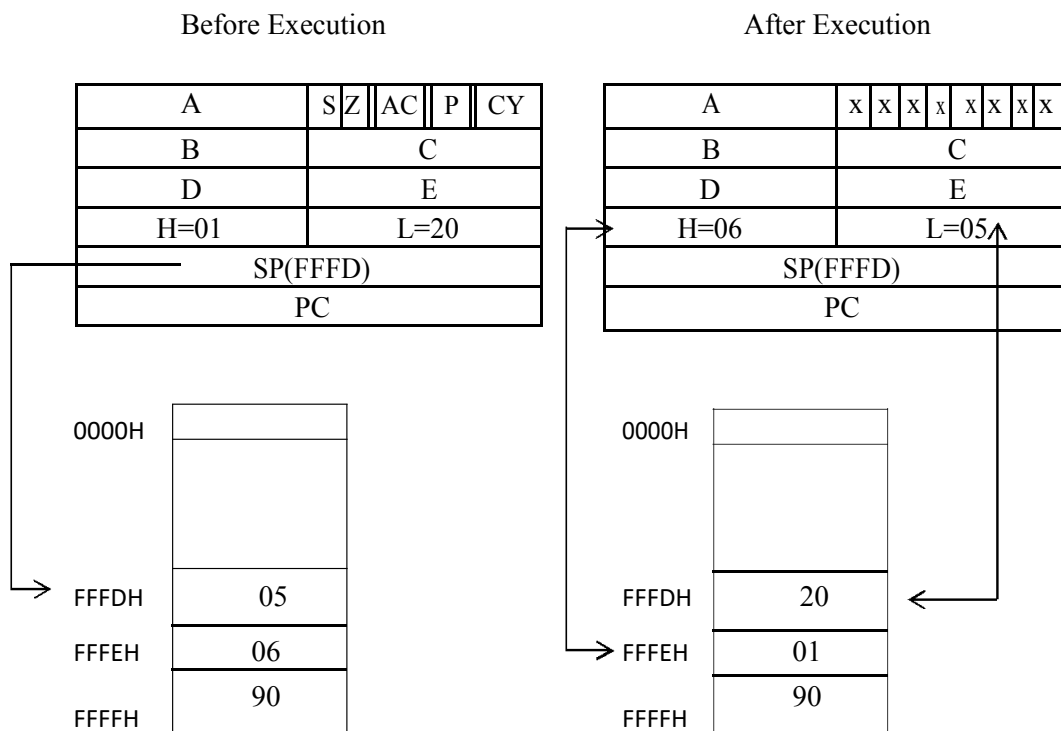
- II. POP Rp $\rightarrow$ e.g., POP D where D is the register pair DE. This instruction pop off the stack contents to register pair. When this instruction is executed, the contents of the memory location pointed by the stack pointer (SP) register are copied to the low order register pair. The SP is incremented by one and the contents of that memory location are copied to the higher order register of register pair. The SP is incremented by 1. The process of POP is exactly opposite to that of PUSH. So, the contents stored by PUSH are taken back using POP instructions.



- III. SPHL: This instruction loads the stack pointer with HL register pair contents. When this instruction is executed, the contents of HL pair are transferred to stack pointer register. H register contents to high order 8-bits and L register contents to low order 8-bits. The contents of H and L are not altered.



- IV. XTHL: This instruction exchanges the HL with top of stack. When this instruction is executed, the contents of L register are exchanged with the stack location pointed by the stack pointer. The contents of H register are exchanged with the next stack location (SP+1). The contents of the stack pointer register are not altered. No flags are modified.



V. NOP: This instruction does no operation

When this instruction is executed no operation is performed. The instruction is fetched and decoded, no operation is executed and microprocessor will go for the next instruction after that. This instruction can be used as a small delay of 4T states or if you have to delete any instruction from your program you can use NOP instead of that instruction.

VI. HLT: This instruction halts and enters wait state.

When this instruction is executed, the microprocessor completes the execution and halts any further execution. The microprocessor enters the halt acknowledge machine cycle and wait states are inserted in every clock period. The address and data bus are placed in high impedance state. The contents of the registers are not modified in this state.

To come out of this state the microprocessor must be interrupted or resetted. If interrupted, the microprocessor executes a call location. Because of call, PC contents are stored on to stack i.e. the address of next instruction. Microprocessor completes ISR(Interrupt Service Routine) and with return instruction it returns back to next instruction after halt.

When the reset signal is applied, PC is loaded with 0000 and execution starts from 0000 address onwards. This instruction is generally used to terminate the program execution. In multiple ending programs at each end this instruction can be used in order to avoid unnecessary jumps to terminate the program.

Branch Group:

The microprocessor executes machine codes in a sequential manner. It goes on executing from one memory location to the next. Branch group of instruction instructs the microprocessor to go to different memory location. It continues executing machine codes from that new location. The address of the new memory location is either specified in the instruction or supplied by the microprocessor or given by extra hardware. The branch group can be divided as:

Jump Instruction:

- I. JMP address
- II. Conditional Jump Instructions
- III. PCHL

Call and Return Instruction

- I. CALL address
- II. Conditional CALL instructions
- III. RET
- IV. Conditional RET instructions

Restart Instructions

- I. RST N

- I. **JMP Address:** This is an unconditional jump instruction where the address is specified in the instruction. When this instruction is executed, it directs the microprocessor about the jump and its address where the microprocessor has to transfer its control. The storing format is Opcode, Low order address and Higher order address.

E.g: JMP C200H where JMP is the opcode, C200H is the 16-bit address.

- II. **Conditional JMP Instructions:** In conditional jump instructions, when the condition is true or satisfied then only the jump is made at the specified address. If the condition is false or not satisfied it will just check and proceed further to execute the next instruction after it.

Different available conditional jumps are as follows:

JC – Jump if carry flag is set.
JNC – Jump if carry flag is reset.
JZ – Jump if zero flag is set.
JNZ – Jump if zero flag is reset.
JP – Jump if positive i.e. sign flag is set.
JM – Jump if negative i.e. sign flag is reset.
JPE – Jump if parity even i.e. parity flag is set.
JPO – Jump if parity odd i.e. parity flag is reset.

Note: No Jump on auxiliary carry flag.

Example JZ C200 : This instruction checks the zero flag. If the zero flag is set, condition is true and the program control is transferred to the address C200H. If zero flag is reset, condition is false and the program control is not transferred instead it will execute the next instruction after JZ C200.

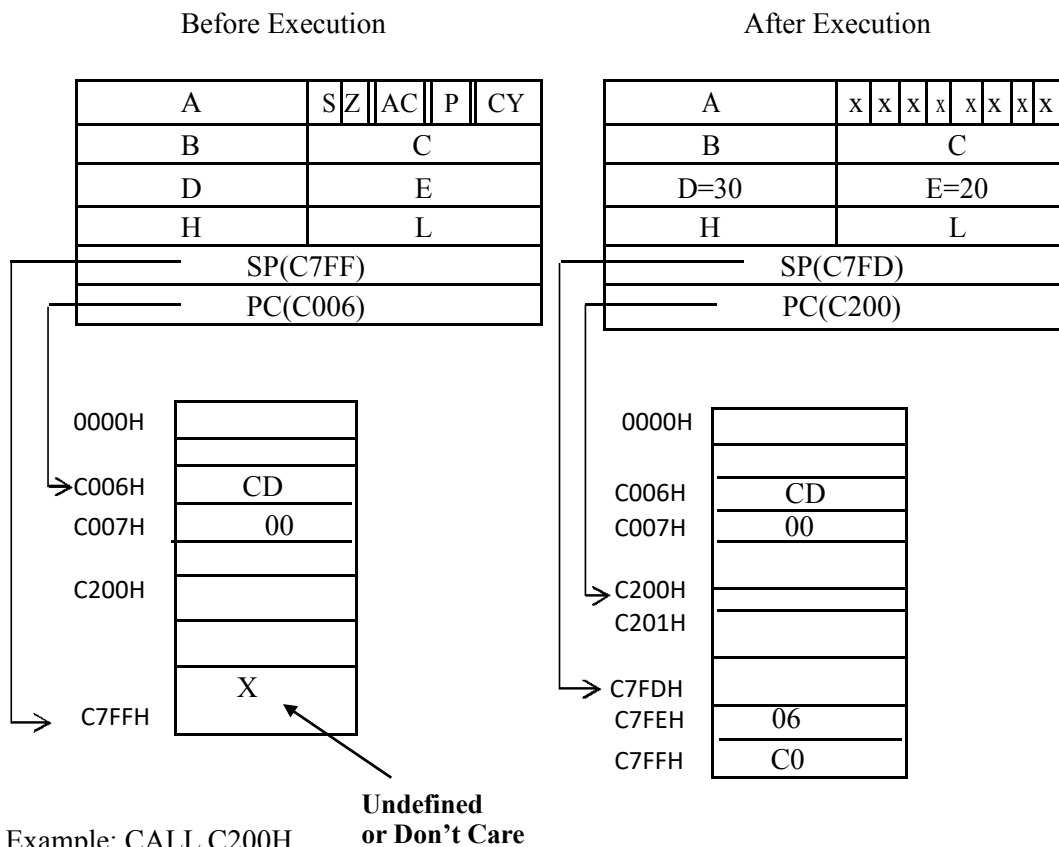
- III. PCHL: This instruction loads program counter with HL contents. The contents of H and L registers are transferred to Program Counter. The H contents to high order 8-bits and L contents to low order 8-bits of program counter. This instruction is equivalent to a 1byte unconditional jump instruction. A program sequence can be changed to any location by simply loading the H and L registers with address and by using this instruction.

Example: LXI H C200H

PCHL

The LXI H instruction initializes HL pair with C200 contents H=C2 and L=00. When PCHL is executed HL contents are transferred to PC and PC=C200, so the next instruction executed will be from C200 address.

- IV. CALL Address: This instruction is an unconditional call subroutine. When this instruction is executed the program sequence is transferred to the address specified in the instruction. Before transferring the sequence the PC contents are stored on to stack i.e. higher order byte and then lower order byte. The call instruction is used to call a subroutine. To return back from the subroutine, the PC contents are stored on to the stack.



Example: CALL C200H

C006H Opcode of Call

C007H 00H

C008H C2H

C009H Next Instruction

When this instruction is executed, program counter contents C009 will be stored on to the stack and microprocessor starts executing instructions from C200H onwards.

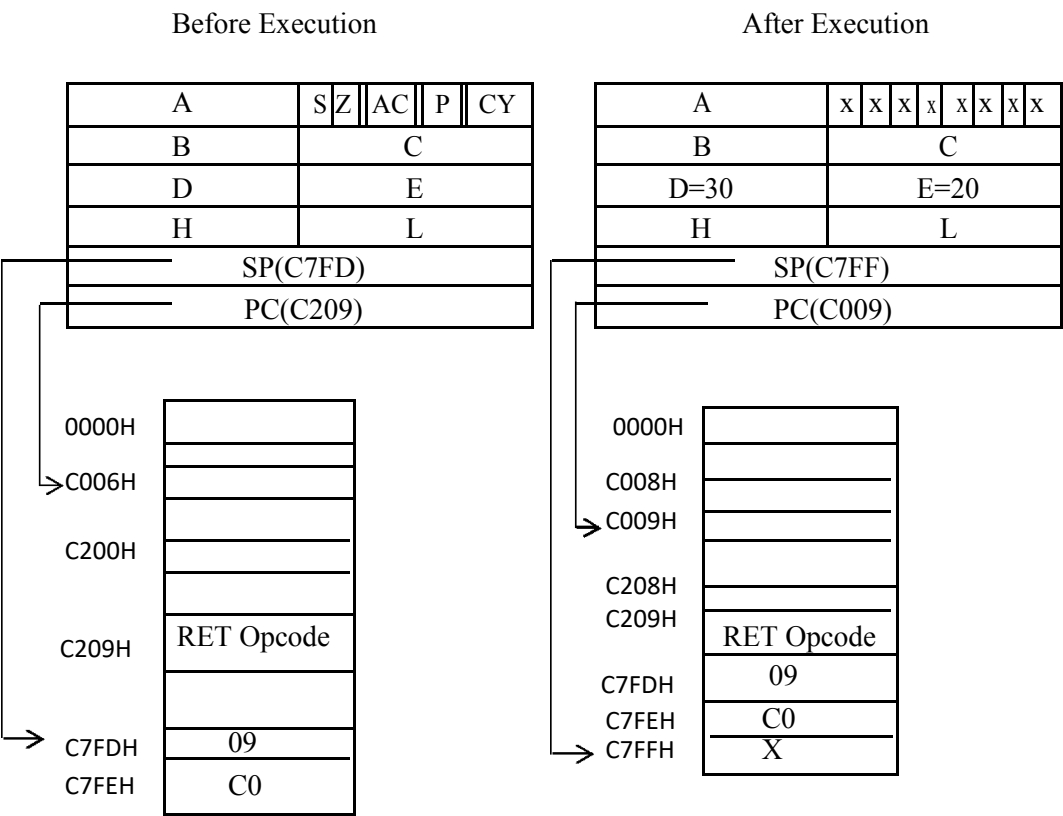
- V. Condition Call Instructions: In conditional CALL instruction, when the condition is true, then a CALL at new address is made. If the condition is false then it will not have a CALL and will proceed for next instruction after it. Different conditional CALL instructions available are:

CC – Call, if carry flag is set.
CNC – Call, if carry flag is reset.
CP – Call, if positive i.e. sign flag is reset.
CM - Call, if minus i.e. sign flag is set.
CPE – Call, if parity even i.e. parity flag is set.
CPO – Call, if parity odd i.e. parity flag is reset.
CZ – Call, if zero flag is set.
CNZ – Call, if zero flag is reset.

Example: CC C200H

Call if carry flag is set, the program written from address C200H onwards will be executed.
If carry flag is reset microprocessor will execute next instruction after CC C200H.

VI. RET: This instruction will return from the subroutine. When this instruction is executed program sequence is transferred from subroutine to the calling program. The return address is taken from stack (where the call instruction has stored its PC contents i.e. return address) this address is loaded in PC and the program execution begins at address taken from stack.



VII. Conditional RET Instructions: In this type of instructions, when the condition is true, then only the RET is made at the address given by the stack. If condition is false, it will proceed further to execute the next instruction after it.

Different conditional RET's available are:

RC – Return from subroutine, if carry flag is set.

RNC – Return from subroutine, if carry flag is reset.

RZ – Return from subroutine, if zero flag is set.

RNZ - Return from subroutine, if zero flag is reset.

RP - Return from subroutine, if positive i.e. if sign flag is reset.

RM - Return from subroutine, if minus i.e. if sign flag is set.

RPO- Return from subroutine, if odd parity i.e. if parity flag is reset.

RPE - Return from subroutine, if even parity i.e. if parity flag is set.

VIII. RST N: These instructions are equivalent to 1 byte CALL instruction at restart location. The contents of the PC are saved on to stack and program jumps to the address. These instructions can be used as software interrupts in a program to transfer program execution to one of the 8 locations depending on which RST instruction is executed.

Restart instructions and its locations are as follows:

Instructions	Restart Locations
RST 0	0000
RST 1	0008
RST 2	0010
RST 3	0018
RST 4	0020
RST 5	0028
RST 6	0030
RST 7	0038

10

Branch Group of Instructions

- More details about program execution
 - PC—Program counter
 - IR—Instruction register
 - W and Z registers
- Unconditional jump instructions
 - JMP a16—Unconditional direct jump
 - PCHL—Unconditional indirect jump
- Conditional jump instructions
 - JNC a16—jump if not Carry
 - JC—jump if carry
 - JNZ—jump if not zero result
 - JZ a16—jump if zero result
 - JPO a16—jump if parity odd
 - JPE a16—jump if parity even
 - JP a16—jump if positive
 - JM a16—jump if minus
- Unconditional call and return instructions
 - Difference between call and jump instructions
 - Conditional call instructions
 - CNC a16—call if no carry
 - CC a16—call if carry
 - CNZ a16—call if not zero result
 - CZ a16—call if zero result
 - CPO a16—call if parity odd
 - CPE a16—call if parity even
 - CP a16—call if positive
 - CM a16—call if minus
 - Conditional return instructions
 - RNC—return if not carry

- *RC—return if carry*
- *RNZ—return if not zero result*
 - *RZ—return if zero result*
 - *RPO—return if parity odd*
 - *RPE—return if parity even*
 - *RP—return if positive*
 - *RM—return if minus*
- *RSTn—restart instructions*
- **Questions**

A total of eight **instruction** types covering 36 instructions will be explained **in** this chapter.

The first part of the chapter gives more details about program execution. Various **instruction** types, such as the conditional and unconditional jump instructions, unconditional and conditional call and return instructions are dealt with **in** the remaining part of the chapter.

■ 10.1 MORE DETAILS ABOUT PROGRAM EXECUTION

The way **8085** executes a simple program (without 'jump' instructions) is as follows. It fetches the first **instruction** from memory and executes it. Then it fetches the next **instruction** from memory and executes it. It continues this way till the halt **instruction** is fetched and executed.

For example, to compute the 2's complement of the number at location F840H and store the result **in** location F850H, the program shown **in** Fig. 10.1 has to be executed.

```
LDA F840H
CMA
INR A
STA F850H
HLT
```

Fig. 10.1
Assembly language program
to compute 2's complement

Let us say, the machine language program is stored starting from memory location F820H as shown **in** Fig. 10.2.

Memory address	Memory contents	
F820H	3AH	
F821H	40H	
F822H	F8H	
F823H	2FH	
F824H	3CH	
F825H	32H	
F826H	50H	
F827H	F8H	
F828H	76H	

Fig. 10.2
Machine language
program to compute
2's complement

To execute the program, 8085 will have to send out the address F820H and fetch the opcode 3AH for the first instruction 'LDA F840H' from memory for execution. In such a case, from which register in 8085 the address value F820H is sent out? The address is sent out from a special purpose register called program counter (PC for short).

10.1.1 PC—PROGRAM COUNTER

PC is a 16-bit register. It contains a memory address. Suppose the PC contents are F820H, then it means that the 8085 desires to fetch the instruction byte at F820H. After fetching the byte at F820H, the PC is automatically incremented by 1. This way 8085 becomes ready to fetch the next byte of the instruction (in case instruction fetch is incomplete), or fetch the next opcode (in case instruction fetch is over).

In the previous program example, first of all PC is loaded with the value F820H. This is done by typing the 'Go' key, then typing 'F820', and finally typing the 'Exec' key. Then the 8085 performs the following action. It sends out F820H the address, which is the content of the PC. From location F820H it receives 3AH, the opcode for LDA. It is received in an 8-bit register called instruction register (IR for short), as shown in Fig. 10.3.

Memory address	Memory contents	Received in 8085 by
F820H	3AH	IR register
F821H	40H	Z register
F822H	F8H	W register
F823H	2FH	IR register
F824H	3CH	IR register
F825H	32H	IR register
F826H	50H	Z register
F827H	F8H	W register
F828H	76H	IR register

Fig. 10.3
Table indicating instruction fetch process

10.1.2 IR—INSTRUCTION REGISTER

IR is a special purpose register, which is used to receive the 8-bit opcode portion of an instruction. It is not accessible to the programmer. What it means is that there are no instructions by which the programmer can load it with values of his choice. For example, instructions like 'MOV IR, D' or 'MVI IR, 45H' are not present in the instruction set of 8085. Thus, IR register is not shown in the programmer's view of 8085.

10.1.3 W AND Z REGISTERS

After the opcode is fetched from location F820H, the PC is incremented to F821H and is sent out as address. Then 8085 receives 40H, the LS byte of data address F840H. It is received in an 8-bit register called Z register. Z is a register, which is used to receive the LS byte portion of the 16-bit address in a 3-byte instruction. It is not accessible to the programmer. Thus, Z register is not shown in the programmer's view of 8085.

Next, PC is incremented to F822H and it receives F8H, the MS byte of data address F840H. It is received in an 8-bit register called W register. W is a register that is used to receive the MS byte portion of a 16-bit address in an instruction. It is not accessible to the programmer.

Now PC is incremented to F823H, but the next instruction fetch does not start as yet. This is because, so far, 8085 has only completely fetched the instruction code for LDA F840H, but it has not yet executed it!

The 8085 executes the instruction LDA F840H, by sending out the address F840H from W and Z registers, and loading the accumulator with the contents of memory location F840H.

Only after the 8085 executes 'LDA F840H' instruction, it sends out the address F823H, which is the content of PC, and receives 2FH, the opcode for CMA. It is received in the IR register. Meanwhile the PC is incremented to F824H. As the complete instruction is received by the 8085, it then executes it using the ALU for the complement operation.

Only after CMA instruction is executed by the 8085, it sends out the address F824H, which is the content of PC, and receives 3CH, the opcode for INR A. It is received in the IR register. Meanwhile the PC is incremented to F825H. As the complete instruction is received by the 8085, it then executes it using the ALU for the increment operation.

Only after INR A instruction is executed by the 8085, it sends out the address F825H, which is the content of PC, and receives 32H, the opcode for STA. It is received in the IR register. Meanwhile the PC is incremented to F826H.

Then F826H from PC is sent out as address and 8085 receives 50H, the LS byte of address F850H. It is received in Z register. Meanwhile PC is incremented to F827H. Then F827H from PC is sent out as address and 8085 receives F8H, the MS byte of address F850H. It is received in W register. Meanwhile PC is incremented to F828H. But the next instruction fetch at F828H does not start as yet. This is because, so far the 8085 has only completely fetched the instruction code for STA F850H, but it has not yet executed it! The 8085 executes the instruction STA F850H, by sending out the address F850H from W and Z registers, and storing the accumulator contents in memory location F850H.

Only after STA F850H instruction is executed by the 8085, it sends out the address F828H, which is the content of PC, and receives 76H, the opcode for HLT. It is received in the IR register. Meanwhile the PC is incremented to F829H. As the complete instruction is received by the 8085, it then executes it, which results in halting the processor.

The role of PC in 8085 for simple programs (without jump instructions) can be summarized as follows:

PC is a 16-bit register. It contains a memory address. Suppose the PC contents are F820H, then it means that the 8085 desires to fetch and execute the instruction starting at F820H. After fetching the instruction starting at F820H, the PC is automatically incremented by 1, 2, or 3 depending on the instruction length. Then the instruction is executed, during which the PC value is not changed.

This way PC will be pointing to the next instruction by the time the current instruction is executed.

■ 10.2 UNCONDITIONAL JUMP INSTRUCTIONS

In simple programs there is a linear program flow from the first instruction to the last instruction in the program. But the real power of a computer exists in its ability to change program flow, and the programs should be written to take advantage of this ability.

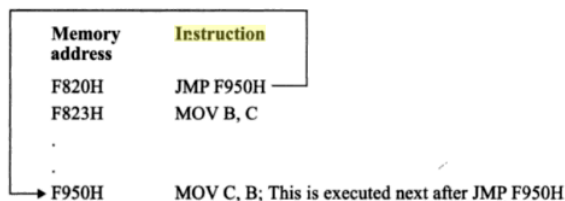
Change in program flow is needed when a sequence of instructions is to be executed repeatedly. It is also needed when it is required to choose between two or more sequences of actions based on some conditions. The branch group of instructions in 8085 effects such a change in program flow. The branch instructions can be used for effecting a forward branch or a backward branch.

An important subgroup of the branch group of instructions is the jump instructions. The jump instructions are classified into:

- Unconditional jump instructions, and
- Conditional jump instructions.

10.2.1 JMP a16—UNCONDITIONAL DIRECT JUMP

JMP is a mnemonic that stands for 'JuMP' and 'a16' stands for any 16-bit address. This instruction is used to jump to the address a16 provided in the instruction. 'JMP F950H' is an example instruction of this type. It is a 3-byte instruction. The result of execution of this instruction is shown below with an example.



In the previous example, after 8085 fetches 'JMP F950H' the PC value would have been automatically incremented by 3 to F823H. Functionally, JMP F950H can be treated as:

JMP F950H = LXI PC, F950H

Thus, execution of JMP F950H results in loading of PC with the value F950H, overwriting the earlier value of F823H. Hence, after execution of JMP F950H, the instruction MOV C, B at memory location F950H will be fetched and executed. In this example, a forward jump was effected. JMP F805H instruction at location F820H, effects a backward jump.

In the instruction set of 8085, 'JMP' is used instead of 'LXI PC', because JMP very explicitly indicates a change in program flow. But, LXI PC just indicates that PC value is changed, but its implied change in program flow may go unnoticed by the programmer.

The result of execution of 'JMP F950H' can be indicated in terms of change in PC value as follows.

	Before	After
(PC)	F820H	F950H

Summary: JMP a16 (3 bytes; JMP F950H; 1 opcode)

10.2.2 PCHL—UNCONDITIONAL INDIRECT JUMP

PCHL is a mnemonic, which stands for 'Load PC with contents of HL'. This instruction is used to jump to the address provided in HL register pair. Thus it is an unconditional indirect jump instruction. Because of its indirect jump feature, it is not very commonly used.

It is a 1-byte instruction compared with the direct jump instruction, which is 3-bytes long. Because of this size advantage, it can be useful for jumping to a frequently used portion of the program.

The result of execution of this instruction is shown below with an example.

Memory address	Instruction
F820H	LXI H, F950H
F823H	PCHL
F824H	MOV B, C
...	
...	
→ F950H	MOV C, B; This is executed next after PCHL

In the previous example, after 8085 fetches 'PCHL' the PC value would have been automatically incremented by 1 to F824H.

But execution of PCHL results in loading of PC with the value F950H, which is the content of HL, overwriting the earlier value of F824H. Hence, after execution of PCHL, the instruction 'MOV C, B' at memory location F950H will be fetched and executed.

The result of execution of 'PCHL' can be indicated in terms of change in PC value as follows.

	Before	After
(HL)	F950H	
(PC)	F823H	F950H

Summary: PCHL (1 byte; PCHL; 1 opcode)

■ 10.3 CONDITIONAL JUMP INSTRUCTIONS

The conditional jump instructions of 8085 perform a jump based on the value of a single flag. The jump takes place based on the value of carry flag, zero flag, parity flag, or sign flag. There is no jump instruction based on the value of auxiliary flag. This is because, generally no one is interested in performing a jump based on this flag!

10.3.1 JNC a16—JUMP IF NOT CARRY

JNC is a mnemonic, which stands for 'Jump if Not Carry', and 'a16' stands for any 16-bit address. This instruction is used to jump to the address a16 provided in the instruction, only if carry flag value is 0. If carry flag value is 1, program flow continues sequentially. 'JNC F950H' is an example instruction of this type. It is a 3-byte instruction. The result of execution of this instruction is shown below with an example.

Memory address	Instruction
F820H	JNC F950H
F823H	MOV B, C; This is executed next after JNC F950H if Cy=1
...	
...	
→ F950H	MOV C, B; This is executed next after JNC F950H if Cy=0

If Cy=0

In the previous example, after 8085 fetches 'JNC F950H' the PC value would have been automatically incremented by 3 to F823H. But JNC F950H instruction execution results in loading of PC with the value F950H, only if Cy flag value is 0.

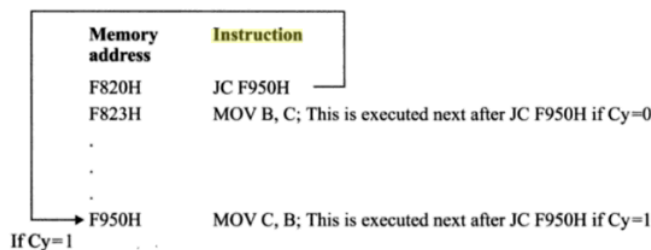
Hence, after execution of JNC F950H, the instruction 'MOV C, B' at memory location F950H will be fetched and executed only if Cy flag value is 0. If Cy flag value is 1, the PC value will remain as F823H, and so the instruction 'MOV B, C' will be executed.

Summary: JNC a16 (3 bytes; JNC F950H; 1 opcode)

10.3.2 JC a16—JUMP IF CARRY

JC is a mnemonic, which stands for 'Jump if Carry'. This instruction is used to jump to the address a16 provided in the instruction, only if carry flag value is 1. If carry flag value is 0, program flow continues sequentially.

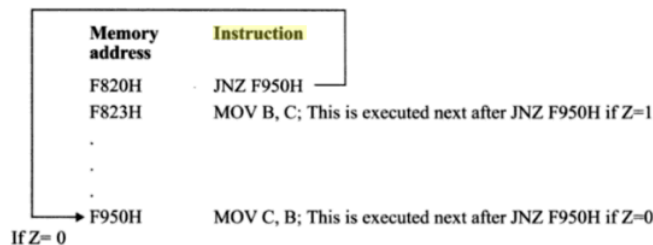
'JC F950H' is an example instruction of this type. It is a 3-byte instruction. The result of execution of this instruction is shown below with an example.



Summary: JC a16 (3 bytes; JC F950H; 1 opcode)

10.3.3 JNZ a16—JUMP IF NOT ZERO RESULT

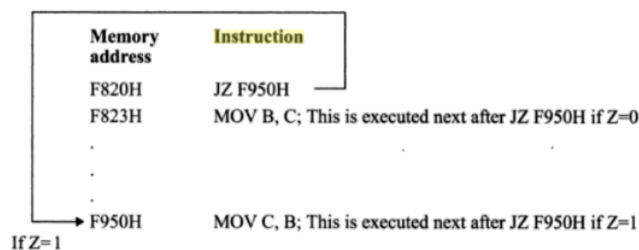
JNZ is a mnemonic, which stands for 'Jump if Not Zero result'. This instruction is used to jump to the address a16 provided in the instruction, only if result is not zero, indicated by Z flag value of 0. If Z flag value is 1, program flow continues sequentially. 'JNZ F950H' is an example instruction of this type. It is a 3-byte instruction. The result of execution of this instruction is shown below with an example.



Summary: JNZ a16 (3 bytes; JNZ F950H; 1 opcode)

10.3.4 JZ a16—JUMP IF ZERO RESULT

JZ is a mnemonic, which stands for 'Jump if Zero result'. This instruction is used to jump to the address a16 provided in the instruction, only if result is zero, indicated by Z flag value of 1. If Z flag value is 0, program flow continues sequentially. 'JZ F950H' is an example instruction of this type. It is a 3-byte instruction. The result of execution of this instruction is shown below with an example.



Summary: JZ a16 (3 bytes; JZ F950H; 1 opcode)

10.3.5 JPO a16—JUMP IF PARITY ODD

JPO is a mnemonic, which stands for 'Jump if Parity Odd'. This instruction is used to jump to the address a16 provided in the instruction, only if parity flag value is 0. If parity flag value is 1, program flow continues sequentially. 'JPO F950H' is an example instruction of this type. It is a 3-byte instruction.

Summary: JPO a16 (3 bytes; JPO F950H; 1 opcode)

10.3.6 JPE a16—JUMP IF PARITY EVEN

JPE is a mnemonic, which stands for 'Jump if Parity Even'. This instruction is used to jump to the address a16 provided in the instruction, only if parity flag value is 1. If parity flag value is 0, program flow continues sequentially.

Summary: JPE a16 (3 bytes; JPE F950H; 1 opcode)

10.3.7 JP a16—JUMP IF POSITIVE

JP is a mnemonic, which stands for 'Jump if Positive'. This instruction is used to jump to the address a16 provided in the instruction, only if sign flag value is 0. If sign flag value is 1, program flow continues sequentially.

Summary: JP a16 (3 bytes; JP F950H; 1 opcode)

10.3.8 JM a16—JUMP IF MINUS

JM is a mnemonic, which stands for 'Jump if Minus'. This instruction is used to jump to the address a16 provided in the instruction, only if sign flag value is 1. If sign flag value is 0, program flow continues sequentially.

Summary: JM a16 (3 bytes; JM F950H; 1 opcode)

■ 10.4 UNCONDITIONAL CALL AND RETURN INSTRUCTIONS

Very often a particular sequence of instructions have to be used at several points in the program. Writing the same sequence of instructions at these various points can be avoided by writing this series of instructions as a subprogram. Such subprograms are also variously termed as subroutines, or procedures.

Whenever the instructions in a subroutine are required to be executed, we branch to the subroutine using the CALL instruction. It is a 3-byte instruction, with 1 byte for the opcode, and 2 bytes for the address of subroutine. CALL stands for 'call a subroutine'. After executing the instructions in the subroutine we return to the next instruction after the CALL instruction in the main program. This is achieved by the execution of the RET (stands for RETURN from subroutine) instruction in the subroutine. RET is a 1-byte instruction.

10.4.1 DIFFERENCE BETWEEN CALL AND JUMP INSTRUCTIONS

The difference between a JMP instruction and a CALL instruction is as follows. If a JMP instruction is executed, we jump to the destination location, and the execution carries on from there, without bothering to come back later to the instruction after the JMP.

If a CALL instruction is executed, we jump to the subroutine, and the execution carries on from there till the RET instruction is executed in the subroutine, and then we come back to the instruction after the CALL in the main program.

The address of the next instruction after the CALL instruction is called the return address. This is the address to which the program flow returns when the RET instruction is executed by the 8085. By the time the 8085 fetches the call instruction, the PC would have been incremented by 3, and will be pointing to the next instruction after call. In other words, PC will have the return address by the time the call instruction is fetched.

In order to facilitate such a return, the CALL instruction will first of all store above the top of stack, the return address. Only then the branch to the subroutine takes place.

Thus a CALL instruction can be visualized as push PC value on the stack, and then branch to the subroutine. Thus CALL F900H can be treated as:

CALL F900H = PUSH PC + JMP F900H

Note that there is no direct instruction like PUSH PC in 8085. As part of the execution of CALL instruction, PUSH PC operation is performed.

The RET instruction, which is at the end of a subroutine, loads the PC with the return address popped out from the top of the stack, so that a branch back to the calling program at the next instruction after CALL is achieved. Thus RET can be treated as:

$$\text{RET} = \text{POP PC}$$

In the instruction set of 8085, 'RET' is used instead of 'POP PC', because RET very explicitly indicates a return back to the calling program. But, POP PC just indicates that PC value is changed, but its implied return to the calling program may go unnoticed by the programmer.

The use of CALL F950H to branch to the subroutine at F950H, and the RET instruction in the subroutine to get back to the main program are illustrated in Fig. 10.4.

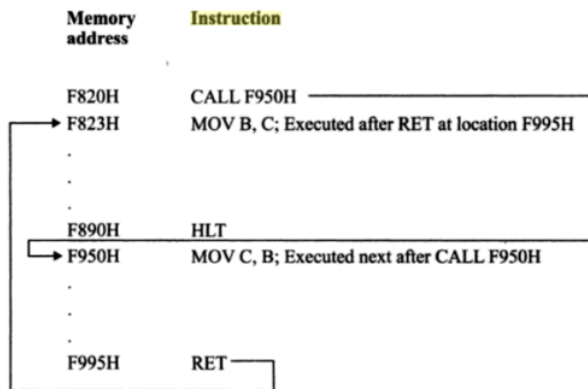


Fig. 10.4
Usage of CALL and RET
instructions

The way stack contents get affected due to execution of 'CALL F950H' is shown below.

	Before	After
(PC)	F820H	F950H
(SP)	FC08H	FC06H
(FC07)	60H	F8H
(FC06)	70H	23H

Note that if the CALL instruction is at location F820H, the return address stored on the top of the stack is F823H.

Summary: CALL a16 (3 bytes; CALL F950H; 1 opcode)

The way in which the stack contents get affected due to execution of RET is shown below.

	Before	After
(PC)	F995H	F823H
(SP)	FC06H	FC08H
(FC06)	23H	
(FC07)	F8H	

Summary: RET (1 byte; RET; 1 opcode)

Using this approach, we can branch any number of times to the subroutine, which is written only once. This saves a lot of memory space for the complete program.

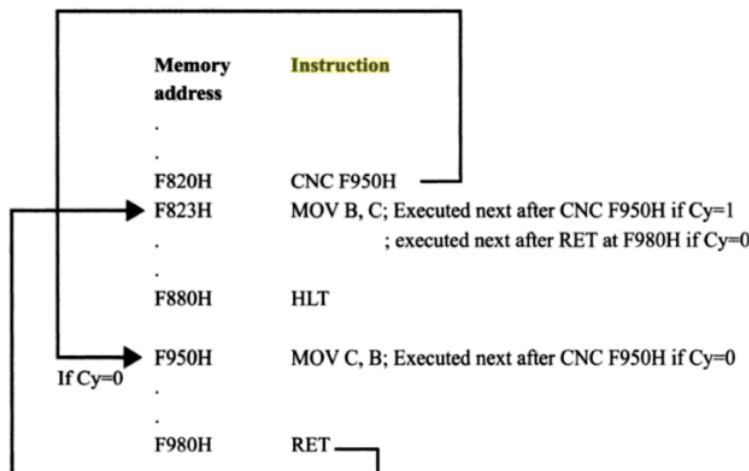
Also usage of subroutines helps in modular design for solving a programming problem. It makes the main program look very simple and compact.

■ 10.5 CONDITIONAL CALL INSTRUCTIONS

The conditional call instructions of 8085 branch to a subroutine based on the value of a single flag. The branch takes place based on the value of Cy flag, Z flag, P flag, or S flag. There is no call instruction based on the value of auxiliary flag. This is because, generally no one is interested in branching to a subroutine based on this flag! The conditional call instructions are 3 bytes in length, 1 byte for the opcode, and another 2 bytes for the subroutine address.

10.5.1 CNC a16—CALL IF NOT CARRY

CNC is a mnemonic, which stands for 'Call if Not Carry'. This instruction is used to branch to the subroutine whose 16-bit address is provided in the instruction, only if Cy flag value is 0. If Cy flag value is 1, program flow continues in the main program sequentially. 'CNC F950H' is an example instruction of this type. It is a 3-byte instruction. The result of execution of this instruction is shown below with an example.



In the previous example, after 8085 fetches 'CNC F950H' the PC value would have been automatically incremented by 3 to F823H. But CNC F950H instruction execution results in saving PC value on the stack and loading of PC with the value F950H, only if Cy flag value is 0.

Hence, after execution of CNC F950H, branch to subroutine at memory location F950H will take place only if Cy flag value is 0. After executing the RET instruction in the subroutine, the program flow will continue with the instruction at F823H in the main program.

If Cy flag value is 1, the PC value will remain as F823H, and so the program flow continues with the instruction MOV B, C in the main program.

Summary: CNC a16 (3 bytes; CNC F950H; 1 opcode)

10.5.2 CC a16—CALL IF CARRY

CC is a mnemonic, which stands for 'Call if Carry'. This instruction is used to branch to the subroutine whose 16-bit address is provided in the instruction, only if Cy flag value is 1. If Cy flag value is 0, program flow continues in the main program sequentially.

Summary: CC a16 (3 bytes; CC F950H; 1 opcode)

10.5.3 CNZ a16—CALL IF NOT ZERO RESULT

CNZ is a mnemonic, which stands for 'Call if Not Zero result'. This instruction is used to branch to the subroutine whose 16-bit address is provided in the instruction, only if result is not zero, indicated by Z flag value of 0. If Z flag value is 1, program flow continues in the main program sequentially.

Summary: CNZ a16 (3 bytes; CNZ F950H; 1 opcode)

10.5.4 CZ a16—CALL IF ZERO RESULT

CZ is a mnemonic, which stands for 'Call if Zero result'. This instruction is used to branch to the subroutine whose 16-bit address is provided in the instruction, only if result is zero, indicated by Z flag value of 1. If Z flag value is 0, program flow continues in the main program sequentially.

Summary: CZ a16 (3 bytes; CZ F950H; 1 opcode)

10.5.5 CPO a16—CALL IF PARITY ODD

CPO is a mnemonic, which stands for 'Call if Parity Odd'. This instruction is used to branch to the subroutine whose 16-bit address is provided in the instruction, only if P flag value is 0. If P flag value is 1, program flow continues in the main program sequentially.

Summary: CPO a16 (3 bytes; CPO F950H; 1 opcode)

10.5.6 CPE a16—CALL IF PARITY EVEN

CPE is a mnemonic, which stands for 'Call if Parity Even'. This instruction is used to branch to the subroutine whose 16-bit address is provided in the instruction, only if P flag value is 1. If P flag value is 0, program flow continues in the main program sequentially.

Summary: CPE a16 (3 bytes; CPE F950H; 1 opcode)

10.5.7 CP a16—CALL IF POSITIVE

CP is a mnemonic, which stands for 'Call if Positive'. This instruction is used to branch to the subroutine whose 16-bit address is provided in the instruction, only if S flag value is 0. If S flag value is 1, program flow continues in the main program sequentially.

Summary: CP a16 (3 bytes; CP F950H; 1 opcode)

10.5.8 CM a16—CALL IF MINUS

CM is a mnemonic, which stands for 'Call if Minus'. This instruction is used to branch to the subroutine whose 16-bit address is provided in the instruction, only if S flag value is 1. If S flag value is 0, program flow continues in the main program sequentially.

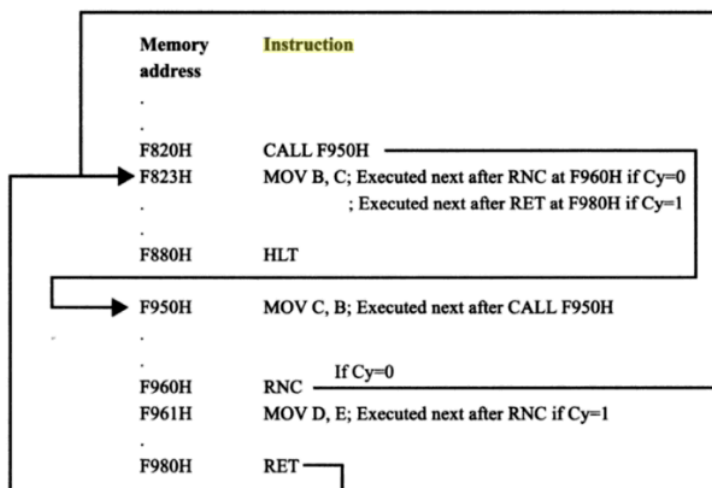
Summary: CM a16 (3 bytes; CM F950H; 1 opcode)

■ 10.6 CONDITIONAL RETURN INSTRUCTIONS

The conditional return instructions of 8085 effect a return to the main program based on the value of a single flag. The return takes place based on the value of Cy flag, Z flag, P flag, or S flag. There is no return instruction based on the value of AC flag. This is because, generally no one is interested in returning to the main program based on this flag! The conditional return instructions are 1 byte in length.

10.6.1 RNC—RETURN IF NOT CARRY

RNC is a mnemonic, which stands for 'Return if Not Carry'. This instruction is used to return to the main program, only if Cy flag value is 0. If Cy flag value is 1, program flow continues in the subroutine sequentially. It is a 1-byte instruction. The result of execution of this instruction is shown below with an example.



In the previous example, when 8085 executes CALL F950H instruction, it saves PC value of F823H on the top of the stack, and branches to the subroutine at F950H.

In the subroutine, after 8085 fetches 'RNC' the PC value would have been automatically incremented by 1 to F961H. But RNC instruction execution results in loading PC value with the return address from the stack top, only if Cy flag value is 0.

Hence, after execution of RNC, return to the main program at memory location F823H will take place only if Cy flag value is 0.

If Cy flag value is 1, the PC value will remain as F961H, and so the program flow continues with the instruction MOV D, E in the subroutine. Finally when the 8085 executes the RET instruction in the subroutine, return to the main program at location F823H takes place.

Summary: RNC (1 byte; RNC; 1 opcode)

10.6.2 RC—RETURN IF CARRY

RC is a mnemonic, which stands for 'Return if Carry'. This instruction is used to return to the main program, only if Cy flag value is 1. If Cy flag value is 0, program flow continues in the subroutine sequentially.

Summary: RC (1 byte; RC; 1 opcode)

10.6.3 RNZ—RETURN IF NOT ZERO RESULT

RNZ is a mnemonic, which stands for 'Return if Not Zero result'. This instruction is used to return to the main program, only if result is not zero, indicated by Z flag value of 0. If Z flag value is 1, program flow continues in the subroutine sequentially.

Summary: RNZ (1 byte; RNZ; 1 opcode)

10.6.4 RZ—RETURN IF ZERO RESULT

RZ is a mnemonic, which stands for 'Return if Zero result'. This instruction is used to return to the main program, only if result is 0, indicated by Z flag value of 1. If Z flag value is 0, program flow continues in the subroutine sequentially.

Summary: RZ (1 byte; RZ; 1 opcode)

10.6.5 RPO—RETURN IF PARITY ODD

RPO is a mnemonic, which stands for 'Return if Parity is Odd'. This instruction is used to return to the main program, only if P flag value is 0. If P flag value is 1, program flow continues in the subroutine sequentially.

Summary: RPO (1 byte; RPO; 1 opcode)

10.6.6 RPE—RETURN IF PARITY EVEN

RPE is a mnemonic, which stands for 'Return if Parity is Even'. This instruction is used to return to the main program, only if P flag value is 1. If P flag value is 0, program flow continues in the subroutine sequentially.

Summary: RPE (1 byte; RPE; 1 opcode)

10.6.7 RP—RETURN IF POSITIVE

RP is a mnemonic, which stands for 'Return if Positive'. This instruction is used to return to the main program, only if S flag value is 0. If S flag value is 1, program flow continues in the subroutine sequentially.

Summary: RP (1 byte; RP; 1 opcode)

10.6.8 RM—RETURN IF MINUS

RM is a mnemonic, which stands for 'Return if Minus'. This instruction is used to return to the main program, only if S flag value is 1. If S flag value is 0, program flow continues in the subroutine sequentially.

Summary: RM (1 byte; RM; 1 opcode)

■ 10.7 RST_n—RESTART INSTRUCTIONS

In this case n has a value from 0 to 7 only. Thus the eight possible RST instructions are RST 0, RST 1, ..., RST 7. These are basically single-byte call instructions. Functionally RST n instruction is:

$$\text{RST } n = \text{CALL } n * 8$$

For example, RST 2 is functionally equivalent to $\text{CALL } 2 * 8 = \text{CALL } 0010\text{H}$. The advantage of RST 2 is that it is only 1 byte, whereas CALL 0010H is 3-byte long. Thus RST instructions are useful for branching to frequently used subroutines.

The reader may wonder how RST n is only 1-byte long, whereas MVI B, d8 is 2-byte long. The reason is that n value is restricted to the range 0–7. So only 3 bits are needed to denote n value, and the other 5 bits in the byte provide the code for RST.

RST 2 is an example instruction of this type. It is a 1-byte instruction. It is functionally same as $\text{CALL } 0010\text{H} = \text{PUSH PC} + \text{JMP } 0010\text{H}$. It causes a branch to subroutine at 0010H. Similarly, RST 3 causes a branch to subroutine at $3 * 8 = 0018\text{H}$. Thus, the subroutine, which starts at location 0010H should not go beyond memory location 0017H. So at the most only eight locations are available for the subroutine, which is too small in general. This limitation is overcome by branching to a subroutine at some other memory location, like F950H. It is achieved by the combination of RST 2 instruction, and JMP F950H instruction at memory location 0010H, as shown in Fig. 10.5.

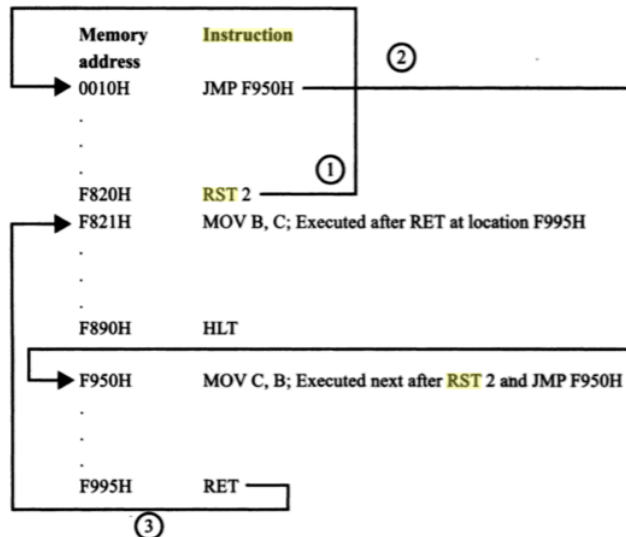


Fig. 10.5
Use of RST 2 to branch to
subroutine at F950H

The way in which the stack contents get affected due to execution of RST 2 is shown below.

	Before	After
(PC)	F820H	0010H
(SP)	FC08H	FC06H
(FC07)	60H	F8H
(FC06)	70H	21H

Note that if the RST 2 instruction is at location F820H, the return address stored on the top of the stack is F821H. Thus PC value is stored on the top of the stack, and a branch to 0010H takes place. But then, because of the JMP F950H at location 0010H, branch to subroutine at F950H finally takes place.

Many times, an I/O port supplies an RST instruction to 8085, when the 8085 is interrupted on INTR pin. Interrupts are discussed in a later chapter.

'RST' stands for 'restart'. They are called restart instructions because, generally one of these instructions is going to transfer control to the monitor program in the microprocessor kit. When a microprocessor kit is switched off, and then switched on, the control is transferred to the monitor program. Thus the action performed by an RST instruction is similar to restarting the kit, and hence the name 'restart' instruction. On the ALS-SDA-85M kit RST 1 instruction transfers control to the monitor program.

Summary: RST n (1 byte; RST 5; 8 opcodes)

The complete role of PC in 8085 can be summarized as follows:

PC is a 16-bit register. It contains a memory address. Suppose the PC contents are F820H. Then it means that the 8085 desires to fetch and execute the instruction starting at F820H. After fetching the instruction starting at F820H, the PC is automatically incremented by 1, 2, or 3 depending on the instruction length. Then the instruction is executed, during which the PC value is not

changed, if the instruction did not belong to the branch group. This way PC will be pointing to the next instruction by the time the current instruction is executed.

If the instruction belonged to the branch group the following actions take place.

- If the current instruction is an unconditional direct jump or call instruction, PC is loaded with the value provided in the JMP or CALL instruction. Thus, a jump is effected.
- If the current instruction is PCHL, PC is loaded with the value present in HL register pair. Thus, a jump is effected.
- If the current instruction is an RST n instruction, PC is loaded with the value $n*8$. Thus, a jump is effected.
- If the current instruction is a conditional jump or call instruction, then
 - PC is loaded with the value provided in the conditional jump or call instruction, if the condition is satisfied. Thus, a jump is effected.
 - PC value is not altered if the condition is not satisfied. In such a case, there will not be a jump.

QUESTIONS

1. Explain the role of Program Counter.
2. What is the purpose of Instruction register?
3. What is the function of W and Z registers?
4. Describe the two types of unconditional jump instructions.
5. List the conditional jump instructions of 8085, and explain any one instruction.
6. What are subroutines? How are they useful?
7. Describe the working of the instructions CALL and RET.
8. Compare JMP and CALL instructions.
9. List the conditional call instructions of 8085, and explain any one instruction.
10. List the conditional return instructions of 8085, and explain any one instruction.
11. Explain the working of RST instructions and their use.
12. Distinguish between the following pairs of instructions:
 - a. RST 5 and CALL 0028H;
 - b. JMP F090H and CALL F090H;
 - c. SPHL and PCHL;
 - d. RET and RPO;
 - e. CALL F090H and CM F090H.
13. RST 5 is 1-byte long, and MVI B, 25H is 2-bytes long. Justify with reason.
14. It is desired to branch to subroutine at location F960H using RST 4 instruction. Explain how you achieve it.
15. Write a 8085 assembly language program, which takes the 8-bit data from memory location X, and counts the number of 1s and 0s in this byte, and stores the result in memory locations X + 1 and X + 2, respectively.

16. Write a 8085 assembly language program, which performs the addition of two 8-bit signed numbers at locations X and X + 1 respectively. If the answer is correct, store it in memory location X + 2. If the answer is wrong, store it in location Y.
(Note: Addition of 40H and 50H gives the wrong answer 90H. It is a wrong answer because 40H and 50H are positive, while 90H is negative.)
17. Write a 8085 assembly language program to generate all Fibonacci numbers, which can be represented using 8 bits, and store them in successive locations starting from location X.